Data_cleaning

May 6, 2026

[1]:
```
import pandas as pd
import numpy as np

print(" Libraries imported successfully!")
```

```
 Libraries imported successfully!
```

[2]:
```
# Load the raw dataset
df = pd.read_csv('Raw_Superstore_Dataset.csv', low_memory=False)

print(" Dataset loaded successfully!")
print(f"Shape: {df.shape}")
print(f"\nFirst 5 Rows:")
print(df.head())
```

```
 Dataset loaded successfully!
Shape: (10194, 21)

First 5 Rows:
   Row ID        Order ID  Order Date   Ship Date       Ship Mode Customer ID  \
0    5848  CA-2015-126725  11/17/2015  11/21/2015  standard class    BS-11665
1    5694  US-2015-138093  10/12/2015  12/16/2015       Std Class    KM-16225
2    5971  CA-2014-141726  20-07-2014   7/22/2014     First Class    CC-12145
3    8965  CA-2016-157280   11/5/2016   11/7/2016     First Class    LW-17125
4    2019  CA-2015-113110   3/19/2015   3/23/2015        Standard    BK-11260

      Customer Name    Segment        Country            City  …  \
0     Brian Stugart   Consumer  United States       San Diego  …
1      Kalyca Meade  Corporate  United States       Baltimore  …
2  Charles Crestani   Consumer  United States       San Diego  …
3    Liz Willingham   Consumer  United States  Virginia Beach  …
4    Berenike Kampe   Consumer  United States  San Bernardino  …

  Postal Code  Region       Product ID         Category Sub-Category  \
0       92105    West  FUR-FU-10001591        Furniture  Furnishings
1       21215    East  OFF-PA-10000143  Office Supplies        Paper
2       92105    West  OFF-PA-10002230  Office Supplies        Paper
3       23464   South  FUR-FU-10003806        Furniture  Furnishings
4       92404    West  OFF-SU-10001165  Office Supplies     Supplies
```

```
                                      Product Name  Sales Quantity  Discount  \
0  Advantus Panel Wall Certificate Holder - 8.5x11   36.6      3.0       0.0
1                  Astroparche Fine Business Paper   26.4      5.0       0.0
2                                       Xerox 1897  19.92      4.0       0.0
3         Tenex Chairmat w/ Average Lip, 45" x 53"    NaN      5.0       0.0
4              Acme Elite Stainless Steel Scissors    NaN      4.0       0.0

    Profit
0  15.3720
1  12.6720
2   9.7608
3  75.6800
4   8.6736

[5 rows x 21 columns]
```

[3]:
```python
print("=" * 50)
print("PROBLEMS IN THE DATASET")
print("=" * 50)

# Missing values
print(f"\n Missing Values:")
print(df.isnull().sum())

# Duplicates
print(f"\n Duplicate Rows: {df.duplicated().sum()}")

# Date formats
print(f"\n Sample Order Dates:")
print(df['Order Date'].head(20).tolist())

# Categorical columns
print(f"\n Ship Mode unique values:")
print(df['Ship Mode'].value_counts())

print(f"\n Category unique values:")
print(df['Category'].value_counts())

print(f"\n Segment unique values:")
print(df['Segment'].value_counts())

print(f"\n Region unique values:")
print(df['Region'].value_counts())

# Sales column
print(f"\n Sample Sales values:")
```

```
print(df['Sales'].head(15).tolist())
```

```
==================================================
PROBLEMS IN THE DATASET
==================================================

  Missing Values:
Row ID             0
Order ID           0
Order Date         0
Ship Date          0
Ship Mode        204
Customer ID        0
Customer Name    306
Segment            0
Country            0
City               0
State            204
Postal Code        0
Region           308
Product ID         0
Category           0
Sub-Category       0
Product Name       0
Sales            503
Quantity         306
Discount         202
Profit           402
dtype: int64

  Duplicate Rows: 196

  Sample Order Dates:
['11/17/2015', '10/12/2015', '20-07-2014', '11/5/2016', '3/19/2015',
'21/03/2015', '5/9/2016', '9/26/2015', '9/1/2016', '11/28/2015', '10/2/2016',
'12/11/2015', '8/12/2017', '9/6/2015', '23/12/2017', '11/9/2015', '5/31/2016',
'5/17/2016', '8/30/2016', '9/10/2017']

  Ship Mode unique values:
Ship Mode
Standard Class    4487
Second Class      1454
First Class       1158
Same Day           405
Std Class          400
standard class     361
STANDARD CLASS     361
Standard           358
```

```
second class       135
SECOND CLASS       132
2nd Class          116
second_class       103
1st Class          102
FIRST CLASS        101
first class         94
first_class         87
same day            40
same_day            36
SAME DAY            35
Sameday             25
Name: count, dtype: int64

  Category unique values:
Category
Office Supplies    4286
Furniture          1512
Technology         1329
OFFICE SUPPLIES     490
office supplies     476
Office Supply       450
office_supplies     439
FURNITURE           187
furnitures          168
TECHNOLOGY          167
Furn.               152
technology          145
furniture           142
tech.               139
Tech                112
Name: count, dtype: int64

  Segment unique values:
Segment
Consumer       3967
Corporate      2307
Home Office    1366
Consumers       336
consumer        334
consumer        331
CONSUMER        325
corporate       202
Corp            197
CORPORATE       191
corp.           178
home office     127
HOME OFFICE     120
```

```
HomeOffice      108
Home_Office     105
Name: count, dtype: int64

 Region unique values:
Region
West        2669
East        2355
Central     1928
South       1336
W            146
west         132
EAST         126
West         122
WEST         121
East         118
E            109
central      108
east         105
Cntrl         92
central       83
CENTRAL       80
S             75
SOUTH         65
south         59
South         57
Name: count, dtype: int64

 Sample Sales values:
['36.6', '26.4', '19.92', nan, nan, '962.08', '856.656', '34.44', nan,
'647.904', '33.96', '69.576', '592.74', '46.32', '28.672']
```

[4]:
```
print("=" * 50)
print("FIX 1 - REMOVE DUPLICATES")
print("=" * 50)

before = df.shape[0]
df = df.drop_duplicates()
after = df.shape[0]

print(f"Rows BEFORE: {before}")
print(f"Rows AFTER: {after}")
print(f"  Removed {before - after} duplicate rows")
```

```
==================================================
FIX 1 - REMOVE DUPLICATES
==================================================
Rows BEFORE: 10194
```

```
Rows AFTER: 9998
  Removed 196 duplicate rows
```

[5]:
```python
print("=" * 50)
print("FIX 2 - STANDARDIZE DATE FORMATS")
print("=" * 50)

print(f"Sample dates BEFORE fix:")
print(df['Order Date'].head(10).tolist())

# Try all possible formats one by one
def parse_mixed_dates(date_series):
    formats = ['%m/%d/%Y', '%d-%m-%Y', '%d/%m/%Y', '%Y/%m/%d', '%Y-%m-%d']
    result = pd.to_datetime(date_series, format=formats[0], errors='coerce')
    for fmt in formats[1:]:
        mask = result.isna() & date_series.notna()
        if mask.sum() == 0:
            break
        result[mask] = pd.to_datetime(date_series[mask], format=fmt,␣
 ↪errors='coerce')
    return result

df['Order Date'] = parse_mixed_dates(df['Order Date'])
df['Ship Date'] = parse_mixed_dates(df['Ship Date'])

print(f"\nNaN dates after parsing: {df['Order Date'].isnull().sum()}")
print(f"\nSample dates AFTER fix:")
print(df['Order Date'].head(10).tolist())

# Drop rows where date truly cannot be parsed
before = df.shape[0]
df = df.dropna(subset=['Order Date'])
after = df.shape[0]
print(f"\nDropped {before - after} unparseable date rows")
print(f"Rows remaining: {after}")

# Extract useful date columns
df['Order Year'] = df['Order Date'].dt.year
df['Order Month'] = df['Order Date'].dt.month
df['Order Month Name'] = df['Order Date'].dt.strftime('%B')
df['Order Quarter'] = df['Order Date'].dt.quarter

print(f"\nYear distribution:")
print(df['Order Year'].value_counts().sort_index())
print(f"\n  Dates fixed successfully!")
```

```
==================================================
FIX 2 - STANDARDIZE DATE FORMATS
```

```
==================================================
Sample dates BEFORE fix:
['11/17/2015', '10/12/2015', '20-07-2014', '11/5/2016', '3/19/2015',
'21/03/2015', '5/9/2016', '9/26/2015', '9/1/2016', '11/28/2015']

NaN dates after parsing: 0

Sample dates AFTER fix:
[Timestamp('2015-11-17 00:00:00'), Timestamp('2015-10-12 00:00:00'),
Timestamp('2014-07-20 00:00:00'), Timestamp('2016-11-05 00:00:00'),
Timestamp('2015-03-19 00:00:00'), Timestamp('2015-03-21 00:00:00'),
Timestamp('2016-05-09 00:00:00'), Timestamp('2015-09-26 00:00:00'),
Timestamp('2016-09-01 00:00:00'), Timestamp('2015-11-28 00:00:00')]

Dropped 0 unparseable date rows
Rows remaining: 9998

Year distribution:
Order Year
2014    1994
2015    2102
2016    2587
2017    3315
Name: count, dtype: int64

 Dates fixed successfully!
```

[8]:
```python
print("=" * 50)
print("FIX 3 - CLEAN SALES COLUMN")
print("=" * 50)

print(f"Sales dtype BEFORE: {df['Sales'].dtype}")
print(f"Sample Sales BEFORE: {df['Sales'].head(10).tolist()}")

# Remove $ symbol and convert to number
df['Sales'] = df['Sales'].astype(str)
df['Sales'] = df['Sales'].str.replace('$', '', regex=False)
df['Sales'] = df['Sales'].str.strip()
df['Sales'] = pd.to_numeric(df['Sales'], errors='coerce')

print(f"\nSales dtype AFTER: {df['Sales'].dtype}")
print(f"Sample Sales AFTER: {df['Sales'].head(10).tolist()}")
print(f"  Sales column cleaned successfully!")
```

```
==================================================
FIX 3 - CLEAN SALES COLUMN
==================================================
Sales dtype BEFORE: float64
```

```
Sample Sales BEFORE: [36.6, 26.4, 19.92, nan, nan, 962.08, 856.656, 34.44, nan,
647.904]

Sales dtype AFTER: float64
Sample Sales AFTER: [36.6, 26.4, 19.92, nan, nan, 962.08, 856.656, 34.44, nan,
647.904]
 Sales column cleaned successfully!
```

[9]:
```python
print("=" * 50)
print("FIX 4 - STANDARDIZE CATEGORY NAMES")
print("=" * 50)

print(f"Category values BEFORE:")
print(df['Category'].value_counts())

# Remove spaces and lowercase
df['Category'] = df['Category'].str.strip().str.lower()

# Map all variations to correct name
category_map = {
    'furniture': 'Furniture',
    'furn.': 'Furniture',
    'furnitures': 'Furniture',
    'technology': 'Technology',
    'tech': 'Technology',
    'tech.': 'Technology',
    'office supplies': 'Office Supplies',
    'office supply': 'Office Supplies',
    'office_supplies': 'Office Supplies',
}
df['Category'] = df['Category'].map(category_map).fillna(df['Category'])

print(f"\nCategory values AFTER:")
print(df['Category'].value_counts())
print(f"  Category cleaned successfully!")
```

```
==================================================
FIX 4 - STANDARDIZE CATEGORY NAMES
==================================================
Category values BEFORE:
Category
Office Supplies    4218
Furniture          1488
Technology         1294
OFFICE SUPPLIES     475
office supplies     466
Office Supply       441
office_supplies     426
```

```
FURNITURE           181
furnitures          165
TECHNOLOGY          164
Furn.               149
technology          142
furniture           141
tech.               137
Tech                111
Name: count, dtype: int64

Category values AFTER:
Category
Office Supplies    6026
Furniture          2124
Technology         1848
Name: count, dtype: int64
  Category cleaned successfully!
```

[10]:
```python
print("=" * 50)
print("FIX 5 - STANDARDIZE SEGMENT NAMES")
print("=" * 50)

print(f"Segment values BEFORE:")
print(df['Segment'].value_counts())

# Remove spaces and lowercase
df['Segment'] = df['Segment'].str.strip().str.lower()

# Map all variations to correct name
segment_map = {
    'consumer': 'Consumer',
    'consumers': 'Consumer',
    'corporate': 'Corporate',
    'corp': 'Corporate',
    'corp.': 'Corporate',
    'home office': 'Home Office',
    'home_office': 'Home Office',
    'homeoffice': 'Home Office',
}
df['Segment'] = df['Segment'].map(segment_map).fillna(df['Segment'])

print(f"\nSegment values AFTER:")
print(df['Segment'].value_counts())
print(f"  Segment cleaned successfully!")
```

```
==================================================
FIX 5 - STANDARDIZE SEGMENT NAMES
==================================================
```

```
Segment values BEFORE:
Segment
Consumer       3896
Corporate      2265
Home Office    1337
Consumers       329
consumer        328
consumer        323
CONSUMER        319
corporate       197
Corp            194
CORPORATE       188
corp.           176
home office     121
HOME OFFICE     118
HomeOffice      105
Home_Office     102
Name: count, dtype: int64

Segment values AFTER:
Segment
Consumer       5195
Corporate      3020
Home Office    1783
Name: count, dtype: int64
 Segment cleaned successfully!
```

[11]:
```python
print("=" * 50)
print("FIX 6 - STANDARDIZE REGION NAMES")
print("=" * 50)

print(f"Region values BEFORE:")
print(df['Region'].value_counts())

# Remove spaces and lowercase
df['Region'] = df['Region'].str.strip().str.lower()

# Map all variations to correct name
region_map = {
    'west': 'West',
    'w': 'West',
    'east': 'East',
    'e': 'East',
    'central': 'Central',
    'cntrl': 'Central',
    'south': 'South',
    's': 'South',
```

```
}
df['Region'] = df['Region'].map(region_map).fillna(df['Region'])

print(f"\nRegion values AFTER:")
print(df['Region'].value_counts())
print(f"  Region cleaned successfully!")
```

```
==================================================
FIX 6 - STANDARDIZE REGION NAMES
==================================================
Region values BEFORE:
Region
West        2613
East        2308
Central     1901
South       1313
W            142
west         128
EAST         124
West         121
WEST         118
East         113
central      106
E            105
east         102
Cntrl         90
central       83
CENTRAL       79
S             73
SOUTH         65
south         58
South         56
Name: count, dtype: int64

Region values AFTER:
Region
West       3122
East       2752
Central    2259
South      1565
Name: count, dtype: int64
  Region cleaned successfully!
```

[13]:
```
print("=" * 50)
print("FIX 7 - STANDARDIZE SHIP MODE NAMES")
print("=" * 50)
```

```python
print(f"Ship Mode values BEFORE:")
print(df['Ship Mode'].value_counts())

# Remove spaces and lowercase
df['Ship Mode'] = df['Ship Mode'].str.strip().str.lower()

# Map all variations to correct name
shipmode_map = {
    'standard class': 'Standard Class',
    'standard': 'Standard Class',
    'std class': 'Standard Class',
    'second class': 'Second Class',
    '2nd class': 'Second Class',
    'second_class': 'Second Class',
    'first class': 'First Class',
    '1st class': 'First Class',
    'first_class': 'First Class',
    'same day': 'Same Day',
    'sameday': 'Same Day',
    'same_day': 'Same Day',
}
df['Ship Mode'] = df['Ship Mode'].map(shipmode_map).fillna(df['Ship Mode'])

print(f"\nShip Mode values AFTER:")
print(df['Ship Mode'].value_counts())
print(f"  Ship Mode cleaned successfully!")
```

```
==================================================
FIX 7 - STANDARDIZE SHIP MODE NAMES
==================================================
Ship Mode values BEFORE:
Ship Mode
Standard Class    4389
Second Class      1429
First Class       1132
Same Day           400
Std Class          394
STANDARD CLASS     358
standard class     356
Standard           354
SECOND CLASS       130
second class       130
2nd Class          114
second_class       102
1st Class          100
FIRST CLASS         99
first class         94
first_class         84
```

```
same day            39
same_day            36
SAME DAY            34
Sameday             24
Name: count, dtype: int64

Ship Mode values AFTER:
Ship Mode
Standard Class    5851
Second Class      1905
First Class       1509
Same Day           533
Name: count, dtype: int64
  Ship Mode cleaned successfully!
```

[14]:
```
print("=" * 50)
print("FIX 8 - HANDLE MISSING VALUES")
print("=" * 50)

print(f"Missing values BEFORE:")
print(df.isnull().sum())

# Numeric columns - fill with median
for col in ['Sales', 'Profit', 'Quantity', 'Discount']:
    median_val = df[col].median()
    missing_count = df[col].isnull().sum()
    df[col] = df[col].fillna(median_val)
    print(f"\n  Filled {missing_count} missing in '{col}' with median:␣
 ↪{median_val:.2f}")

# Categorical columns - fill with mode
for col in ['Customer Name', 'Region', 'Ship Mode', 'State']:
    mode_val = df[col].mode()[0]
    missing_count = df[col].isnull().sum()
    df[col] = df[col].fillna(mode_val)
    print(f"  Filled {missing_count} missing in '{col}' with mode: {mode_val}")

print(f"\nMissing values AFTER:")
print(df.isnull().sum())
print(f"\n  Missing values handled successfully!")
```

```
==================================================
FIX 8 - HANDLE MISSING VALUES
==================================================
Missing values BEFORE:
Row ID                0
Order ID              0
Order Date            0
```

```
Ship Date             0
Ship Mode           200
Customer ID           0
Customer Name       301
Segment               0
Country               0
City                  0
State               200
Postal Code           0
Region              300
Product ID            0
Category              0
Sub-Category          0
Product Name          0
Sales               510
Quantity            300
Discount            199
Profit              395
Order Year            0
Order Month           0
Order Month Name      0
Order Quarter         0
dtype: int64

  Filled 510 missing in 'Sales' with median: 54.69

  Filled 395 missing in 'Profit' with median: 8.67

  Filled 300 missing in 'Quantity' with median: 3.00

  Filled 199 missing in 'Discount' with median: 0.20
  Filled 301 missing in 'Customer Name' with mode: William Brown
  Filled 300 missing in 'Region' with mode: West
  Filled 200 missing in 'Ship Mode' with mode: Standard Class
  Filled 200 missing in 'State' with mode: California

Missing values AFTER:
Row ID              0
Order ID            0
Order Date          0
Ship Date           0
Ship Mode           0
Customer ID         0
Customer Name       0
Segment             0
Country             0
City                0
State               0
```

```
Postal Code         0
Region              0
Product ID          0
Category            0
Sub-Category        0
Product Name        0
Sales               0
Quantity            0
Discount            0
Profit              0
Order Year          0
Order Month         0
Order Month Name    0
Order Quarter       0
dtype: int64

 Missing values handled successfully!
```

[15]:
```python
print("=" * 50)
print("FIX 9 - REMOVE OUTLIERS")
print("=" * 50)

print(f"Rows BEFORE: {df.shape[0]}")
print(f"\nRanges BEFORE:")
print(f"Sales:    {df['Sales'].min():.2f} to {df['Sales'].max():.2f}")
print(f"Profit:   {df['Profit'].min():.2f} to {df['Profit'].max():.2f}")
print(f"Quantity: {df['Quantity'].min()} to {df['Quantity'].max()}")
print(f"Discount: {df['Discount'].min():.2f} to {df['Discount'].max():.2f}")

# Remove invalid Sales
df = df[df['Sales'] > 0]
df = df[df['Sales'] < 50000]

# Remove invalid Profit
df = df[df['Profit'] > -5000]
df = df[df['Profit'] < 50000]

# Remove invalid Quantity
df = df[df['Quantity'] > 0]

# Remove invalid Discount
df = df[(df['Discount'] >= 0) & (df['Discount'] <= 1)]

print(f"\nRanges AFTER:")
print(f"Sales:    {df['Sales'].min():.2f} to {df['Sales'].max():.2f}")
print(f"Profit:   {df['Profit'].min():.2f} to {df['Profit'].max():.2f}")
print(f"Quantity: {df['Quantity'].min()} to {df['Quantity'].max()}")
```

```
print(f"Discount: {df['Discount'].min():.2f} to {df['Discount'].max():.2f}")

print(f"\nRows AFTER: {df.shape[0]}")
print(f"  Outliers removed successfully!")
```

```
==================================================
FIX 9 - REMOVE OUTLIERS
==================================================
Rows BEFORE: 9998

Ranges BEFORE:
Sales:    -500.00 to 999999.00
Profit:   -99999.00 to 999999.00
Quantity: -5.0 to 14.0
Discount: -0.10 to 2.00

Ranges AFTER:
Sales:    0.44 to 22638.48
Profit:   -3839.99 to 8399.98
Quantity: 1.0 to 14.0
Discount: 0.00 to 0.80

Rows AFTER: 9733
  Outliers removed successfully!
```

[17]:
```
print("=" * 50)
print("  FINAL CHECK")
print("=" * 50)

print(f"Final Shape: {df.shape}")

print(f"\nMissing Values (all should be 0):")
print(df.isnull().sum())

print(f"\nCategory values:")
print(df['Category'].value_counts())

print(f"\nSegment values:")
print(df['Segment'].value_counts())

print(f"\nRegion values:")
print(df['Region'].value_counts())

print(f"\nShip Mode values:")
print(df['Ship Mode'].value_counts())

print(f"\nYear distribution:")
```

```python
print(df['Order Year'].value_counts().sort_index())

print(f"\nBasic Statistics:")
print(df[['Sales','Profit','Quantity','Discount']].describe())

# Save clean dataset
df.to_csv('Clean_Superstore.csv', index=False)

print("\n" + "=" * 50)
print("  Clean dataset saved as 'Clean_Superstore.csv'")
print("=" * 50)
```

```
==================================================
 FINAL CHECK
==================================================
Final Shape: (9733, 25)

Missing Values (all should be 0):
Row ID              0
Order ID            0
Order Date          0
Ship Date           0
Ship Mode           0
Customer ID         0
Customer Name       0
Segment             0
Country             0
City                0
State               0
Postal Code         0
Region              0
Product ID          0
Category            0
Sub-Category        0
Product Name        0
Sales               0
Quantity            0
Discount            0
Profit              0
Order Year          0
Order Month         0
Order Month Name    0
Order Quarter       0
dtype: int64

Category values:
Category
Office Supplies    5869
```

```
Furniture          2065
Technology         1799
Name: count, dtype: int64

Segment values:
Segment
Consumer       5046
Corporate      2938
Home Office    1749
Name: count, dtype: int64

Region values:
Region
West       3331
East       2674
Central    2215
South      1513
Name: count, dtype: int64

Ship Mode values:
Ship Mode
Standard Class    5892
Second Class      1847
First Class       1475
Same Day           519
Name: count, dtype: int64

Year distribution:
Order Year
2014    1929
2015    2046
2016    2535
2017    3223
Name: count, dtype: int64

Basic Statistics:
              Sales       Profit     Quantity     Discount
count   9733.000000  9733.000000  9733.000000  9733.000000
mean     217.741002    28.149159     3.766156     0.157117
std      606.829770   213.527084     2.198853     0.204508
min        0.444000 -3839.990400     1.000000     0.000000
25%       18.312000     1.932000     2.000000     0.000000
50%       54.686000     8.671000     3.000000     0.200000
75%      194.352000    27.435200     5.000000     0.200000
max    22638.480000  8399.976000    14.000000     0.800000

==================================================
 Clean dataset saved as 'Clean_Superstore.csv'
```

==========